Artificial Intelligence: Swarm Intelligence

By: Alvin Nathaniel Tjondrowiguno
(with some additional material from Greg)

Previously: Genetic Algorithm

- Chromosomes
- Fitness function
- Selection
- Crossover
- Mutation

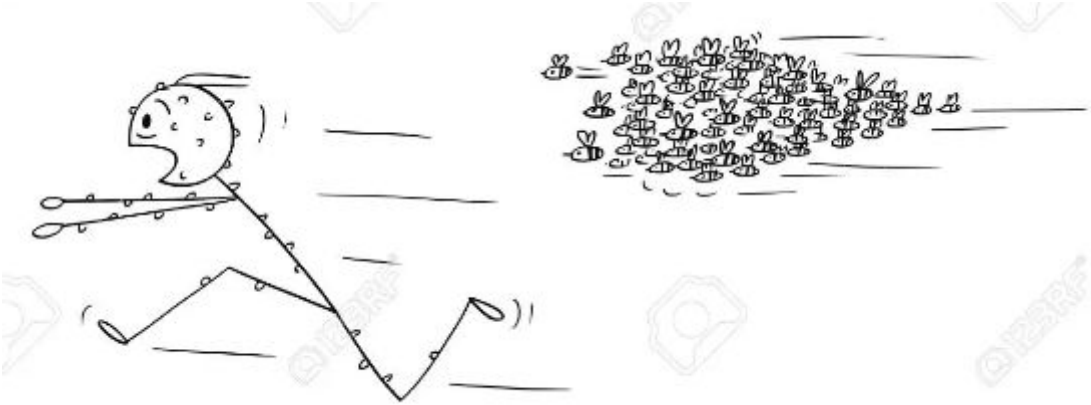
Swarm Intelligence

What is Swarm Intelligence?

- An artificial intelligence (AI) technique based on the **collective behavior** in **decentralized**, **self-organized** systems
  - Generally made up of agents who interact with each other and the environment
  - **No centralized** control structures
- Based on group behavior found in nature

Examples of Swarms in Nature:

- Classic Example: **Swarm of Bees**
- Can be extended to other similar systems:
  - Ant colony
    - Agents: ants
  - Flock of birds
    - Agents: birds
  - Traffic
    - Agents: cars
  - Crowd
    - Agents: humans
  - Immune system
    - Agents: cells and molecules



Why Swarm?

- Animals have few brain cells
  - **Individually**, they cannot really do much
- **Collectively**, however, they perform complex things with great success
  - Fly in a general direction without bumping into each other
  - Communicate with each other through the environment
  - Find sources of food for the entire population
  - Find shortest routes to food sources
  - Carry items several times their body weights
  - Build complex nests
  - etc.

Swarm Intelligence Algorithms

Two popular Swarm Intelligence algorithms:

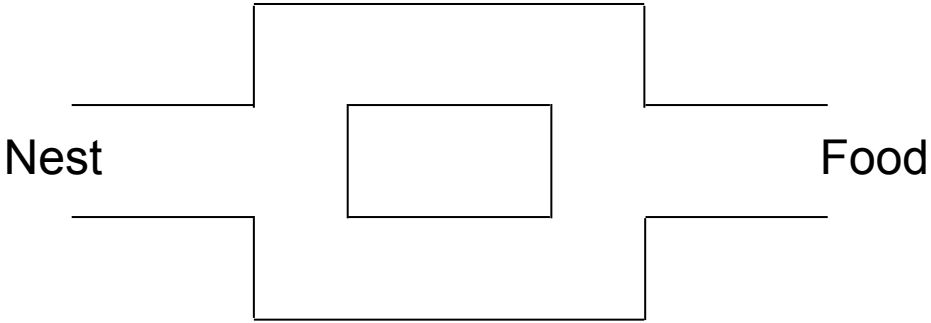
- Ant Colony Optimization (ACO)
- Particle Swarm Optimization (PSO)

Ant Colony Optimization

Why Ants?

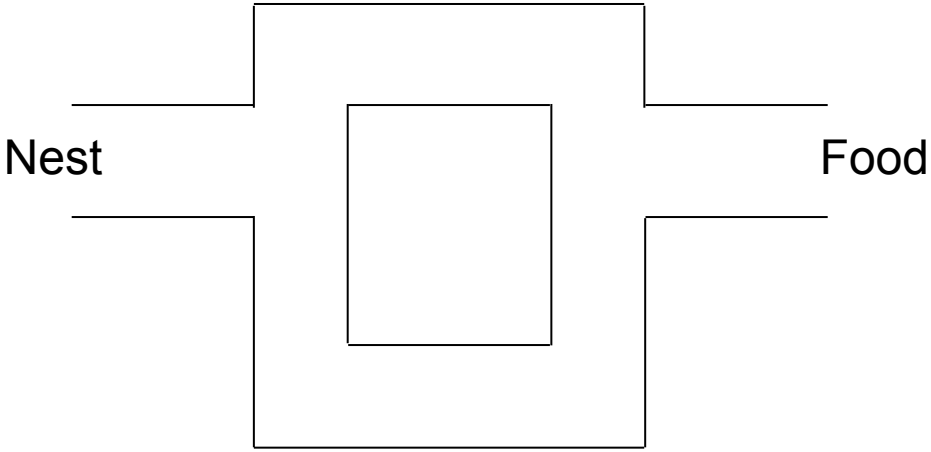
- The concept of **Stigmergy**:
  “An indirect communication via interaction with the environment”
- When they move, ants lay **pheromones** so that other ants can follow the trail
  - Pheromones are the ants’ way of telling each other “I went this way”
  - When faced with a branch in the path between two alternative pathways, ants choose the one with the **stronger pheromone**
  - Pheromones gradually **evaporate** over time
  - The more favorable a certain path is for the colony, the more ants will follow it, the stronger the pheromones are on that path

Two Bridges Experiment



- Paths taken by ants to reach the food source from the nest are observed
- At first, both bridges are used **equally**
- After a while, only **one** bridge is used by all ants!

Two Bridges Experiment



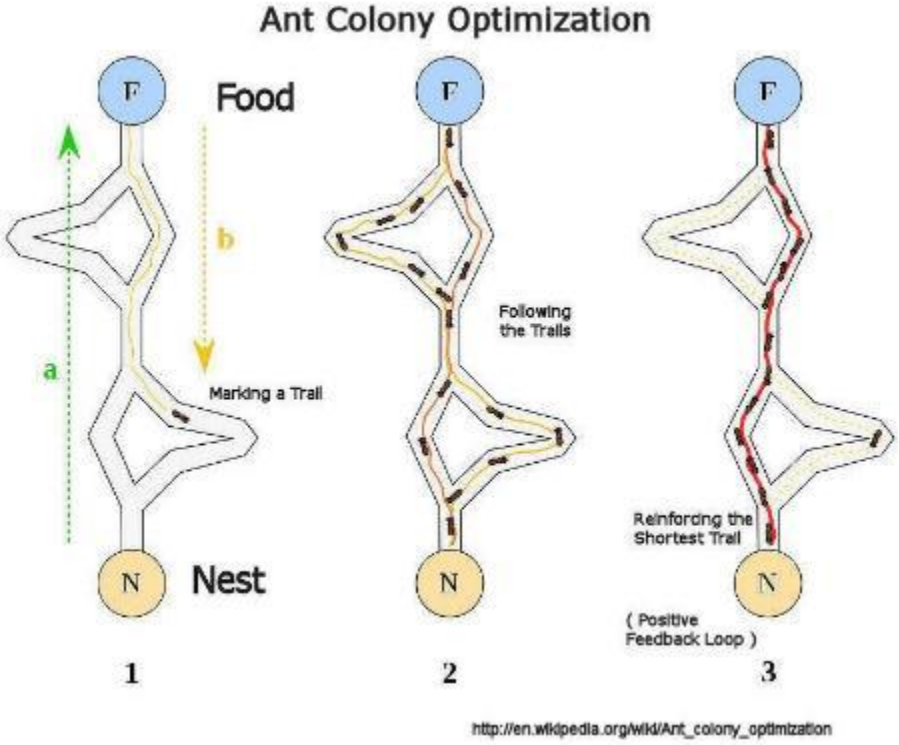
- When the length of the bridges are different, at the end all ants use the **shorter** bridge!

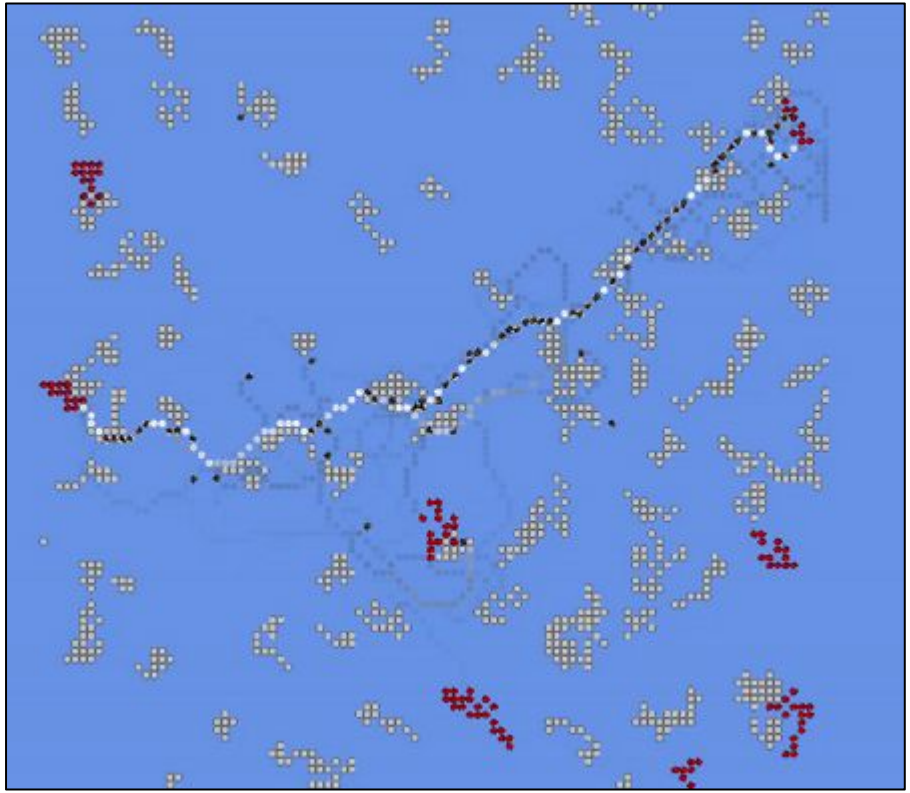
Two Bridges Experiment

How do the ants choose the **shorter** bridge?

- Initially, they travel between bridges **randomly**
- Ants that travels through the shorter bridge **return faster**
- More and more ants choose the shorter bridge because the pheromone concentration is **higher** on the shorter bridge, thus generating **even more** pheromone
- And so on...

How do the ants choose the shorter path?



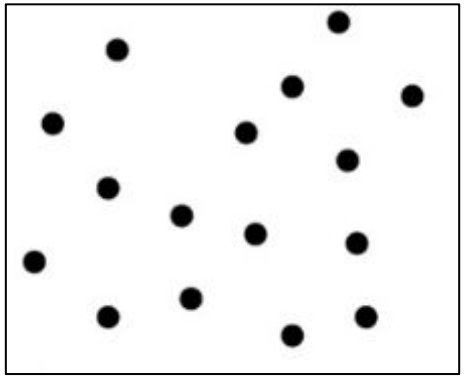


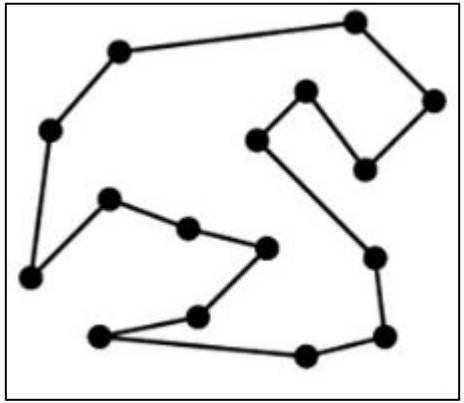
Problem Formulation

- For ACO, the problem is formulated as a **network of paths**
  - Graph representation (vertices, edges, and edge costs)
  - Solutions are **paths** going through the graph, which are constructed incrementally by ants moving through the graph
  - Objective function to compare one solution to another
- A perfect problem example is the **Traveling Salesman Problem**

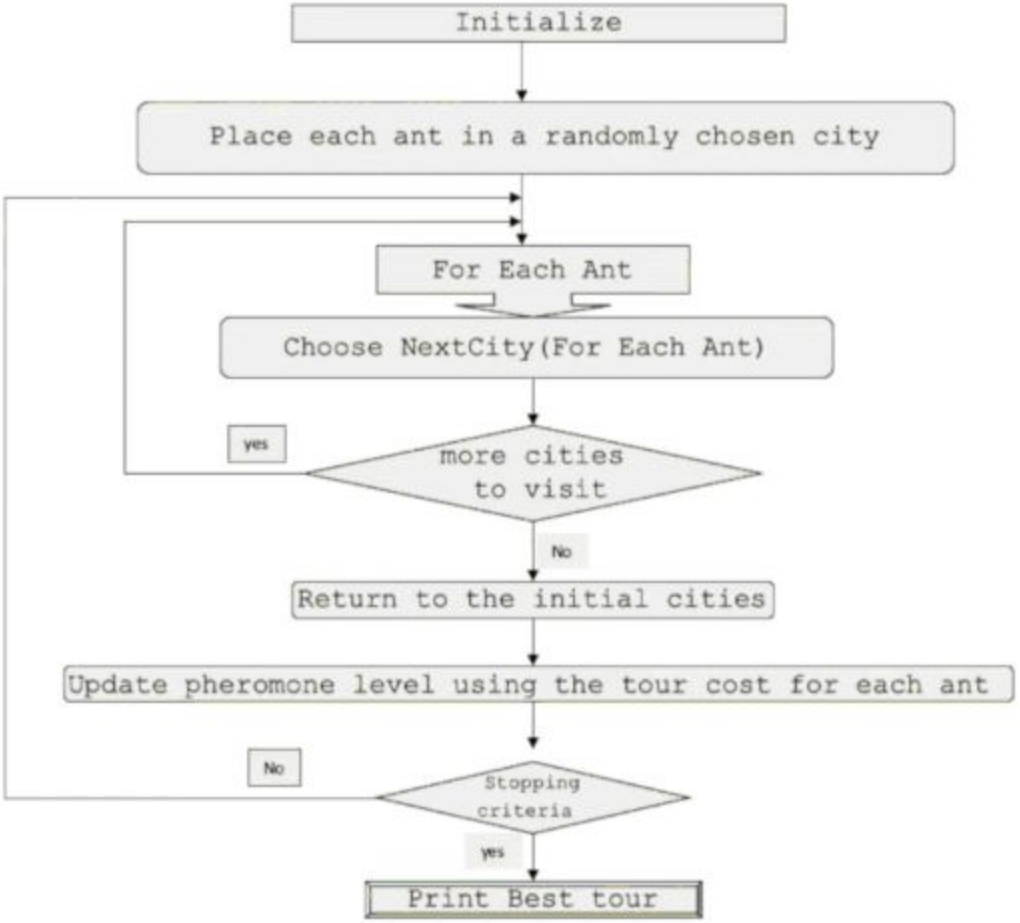
Traveling Salesman Problem (TSP)

- Given:
  - A list of “cities”
  - Distances between all pairs of cities
- Find a **shortest** possible route that:
  - Starts in a certain origin city
  - Visits every single city once
  - Returns to the origin city





Algorithm for TSP(AS)



Ant Colony Optimization

General outline:

- Start all ants in **arbitrarily random** nodes
- For each iteration, **move** all ants to another node based on:
  - **Neighborhood** of the nodes (in case of TSP, every other **unvisited** city is a neighbor)
  - **Pheromone** trails on the path between the nodes
  - **Attractiveness** of the paths (in case of TSP, the inverse distance)
- When all ants have finished their routes through the graph, **update** the pheromones on the paths based on:
  - All pheromones **evaporate**
  - Pheromones on paths that were traveled by ants are **reinforced** based on the **value** of the path
- If **stopping condition** is not satisfied, repeat

Ant Colony Optimization

- Ants choose paths **randomly** according to a probability distribution $\mathbf{p_{ij}}$

$$p_{ij}^{(t)} = \frac{\tau_{ij}^{\alpha}\,\eta_{ij}^{\beta}}{\sum\limits_{k \in \mathcal{N}_i(t)} \tau_{ik}^{\alpha}\,\eta_{ik}^{\beta}} \quad \text{for } j \in \mathcal{N}_i(t)$$

- p_{ij} : Probability of moving from node **i** to node **j**
- τ_{ij} *(tau)*: Pheromone level on the path from **i** to **j**
- η_{ij} *(eta)*: Attractiveness of the path from **i** to **j** (or $1/d_{ij}$ where d_{ij} is the distance)
- $N_i(t)$: Neighbors of **i** at time **t** (unvisited nodes reachable from **i** at time **t**)
- α *(alpha)*, β *(beta)*: Adjustable parameters; empirically $\alpha = 1$, $\beta \in [2, 5]$

Ant Colony Optimization

- Pheromones are updated after **μ *(mu)*** ants have finished their routes

$$\tau_{ij} := \rho\,\tau_{ij} + \sum_{k=1}^{\mu} \Delta\tau_{ij}(k)$$

- τ_{ij} : Pheromone level on the path from **i** to **j**
- $\Delta\tau_{ij}(k)$: Objective value of a route (a solution) traveled by ant **k**, **IF** the route traveled by ant **k** involves the path from **i** to **j**

  For TSP, $\Delta\tau_{ij}(k)$ = 1/(total route length of ant **k**)
- ρ *(rho)*: Evaporation rate; ρ ∈ [0, 1]

Applications of ACO

- Ant Colony Optimization can be applied to many other problems, as long as the problem can be formulated as a **network of paths**
- Example applications:
  - Transportation problem
  - Assignment problem
    - Quadratic assignment problem (QAP)
    - Generalized assignment problem (GAP)
    - Frequency assignment problem (FAP)
    - Redudancy allocation problem
  - Scheduling problem
    - Job-shop scheduling problem (JSP)
    - Open-shop scheduling problem (OSP)
    - Resource-constrained project scheduling problem (RCPSP)
    - Group-shop scheduling problem (GSP)
  - Network routing problem

Particle Swarm Optimization

Scenario

- PSO simulates the behaviors of **bird flocking**.

- **Scenario:** a group of birds are randomly searching food in an area.
  - Only one piece of food in the area being searched.
  - The birds don't know where is the food.
  - Eventually they know how far the food is in each iteration.
  - **What's the best strategy to find the food ?**

Particle Swarm Optimization

- Using a collection of agents (**particles**) that constitutes a **swarm** to look for a **best** solution by "***moving around***" in the search space
- Inspired by natural swarms: Flocks of birds, swarms of bees, etc.
- The search space is a **real number** (**continuous**) space
- Each **particle's position (vector)** represents a **possible solution**
- Each particle *moves around* the search space based on:
  - Current movements (position, velocity => vectors)
  - Personal experience
  - Social experience

Particle Swarm Optimization

General Outline:

- Initialize particles with random **position** and **velocity**
- Calculate and set initial **personal best** value and **global best** value
- Until **stopping condition** is satisfied, repeat:
  - Update the **velocity** of each particle based on:
    - Current velocity
    - Personal best value
    - Global best value
  - Update the **position** of each particle based on:
    - Current position
    - Updated velocity

Algorithm 1.1 Basic PSO

1. Initialise the particles' positions and velocities randomly. Initialise each particle's personal best (*Pb*) and the swarm's global best (*Gb*);
2. Update the particles' positions using Eq. 2 and Eq. 3:
3. $$v_{lq}^{t} = wv_{lq}^{t-1} + c_1 rnd_{11}(Pb_{lq}^{t-1} - U_{lq}^{t-1}) + c_2 rnd_{12}(Gb_{lq}^{t-1} - U_{lq}^{t-1}) \quad (2)$$
4. $$U_{lq}^{t} = U_{lq}^{t-1} + min\{Vmax, v_{lq}^{t}\} \quad (3)$$
5. Update *Pb* and *Gb*;
6. Update the inertia-weight, *w*, randomly;
7. If terminating criteria are met, stop and report *Gb*; else go to step 2.

Where:

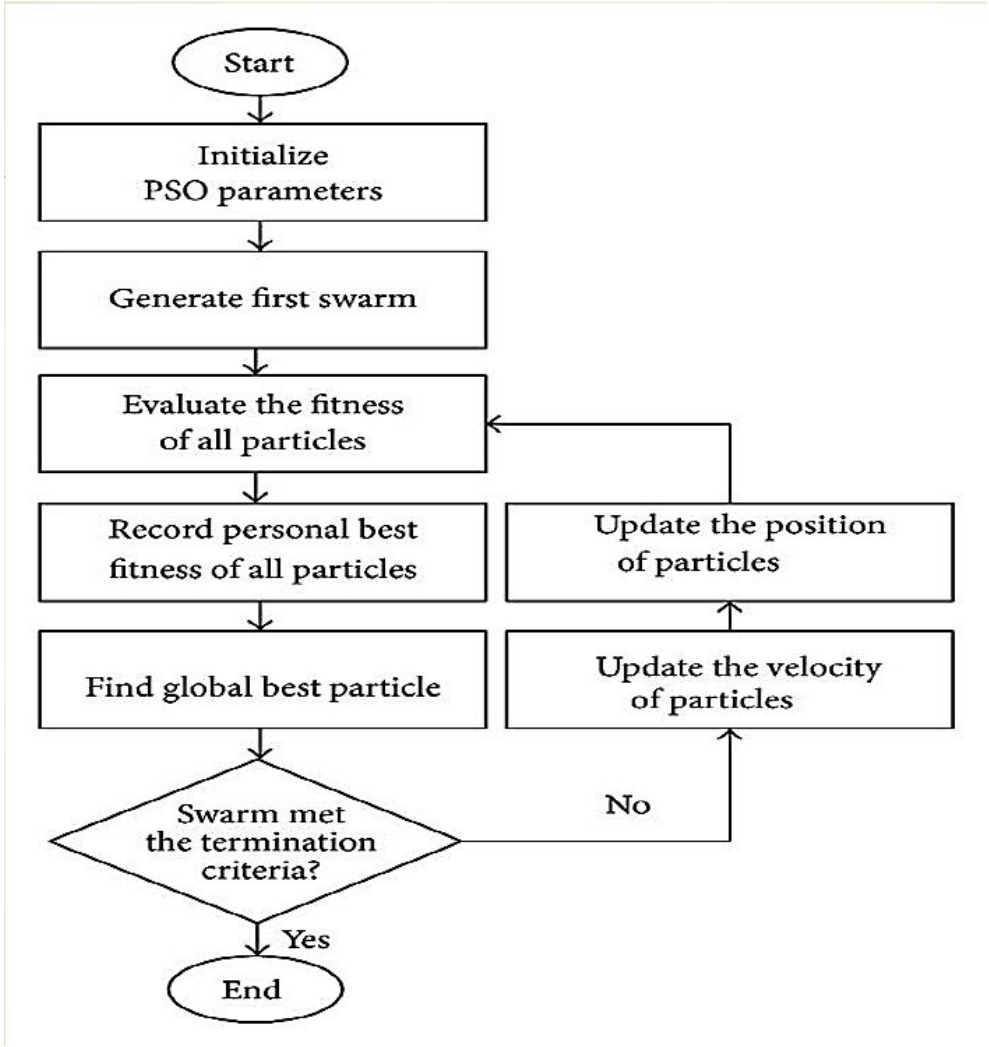
U_{lq}^{t}, Pb_{lq}^{t}, Gb_{lq}^{t} and v_{lq}^{t} : The current position, personal best position *Pb*, global best position Gb, and the velocity v of particle l at dimension q and iteration t;

rnd_{11} and rnd_{12}: Random values from 0 to 1;

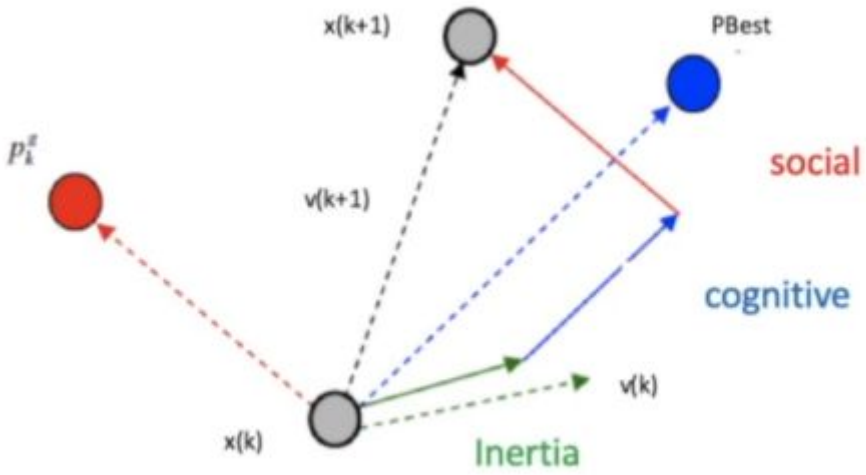
c_1 and c_2 : The weights for regulating the influence of *Pb* and *Gb*;

w is the inertia weight for balancing personal and global exploration abilities of the swarm

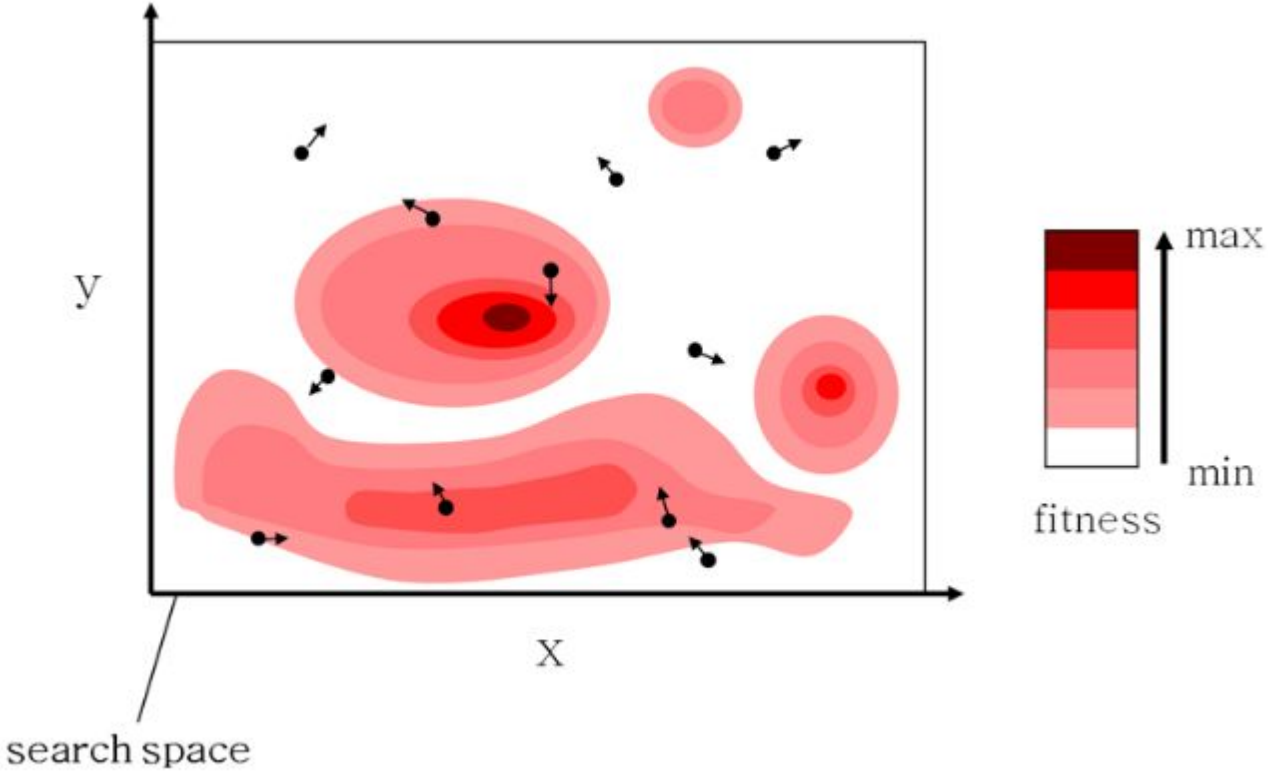
Vmax : The maximum velocity of the particle in order to stop it from moving beyond limitation


Start
Initialize PSO parameters
Generate first swarm
Evaluate the fitness of all particles
Record personal best fitness of all particles
Find global best particle
Swarm met the termination criteria?
No
Yes
End
Update the position of particles
Update the velocity of particles

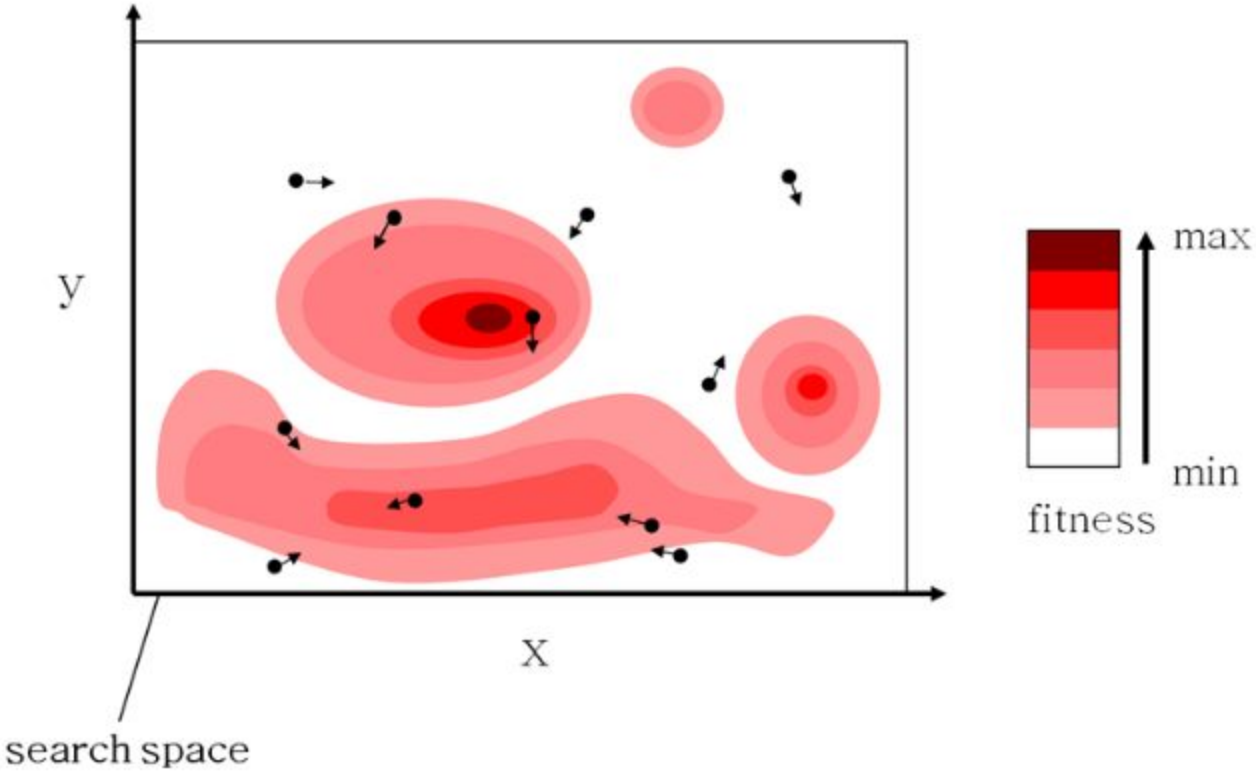
Example



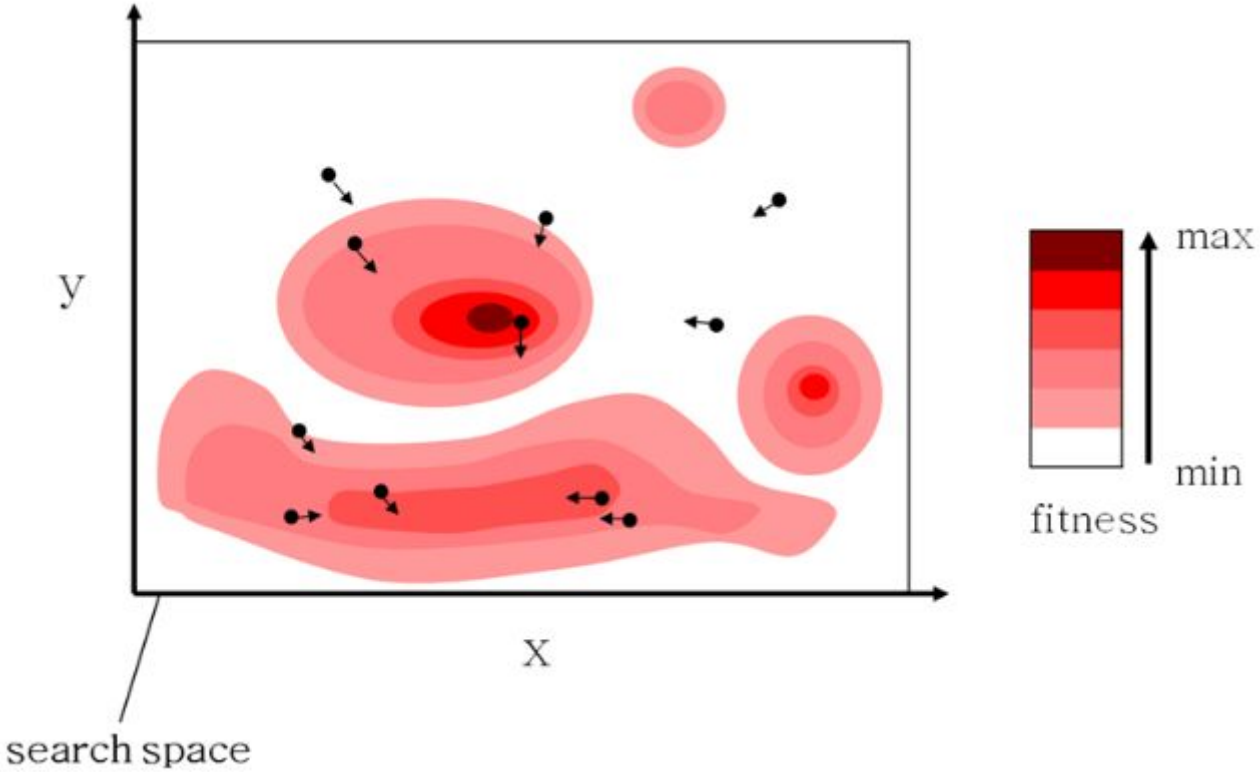
Visualization



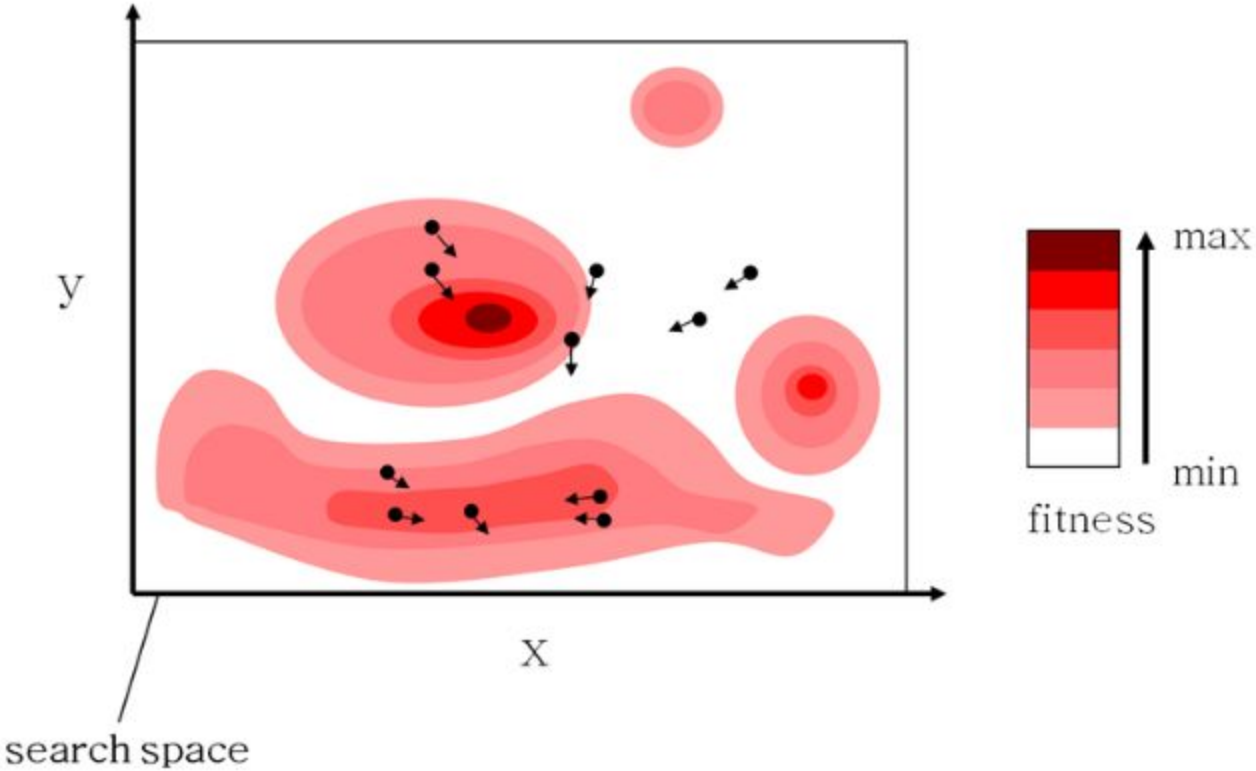
Visualization



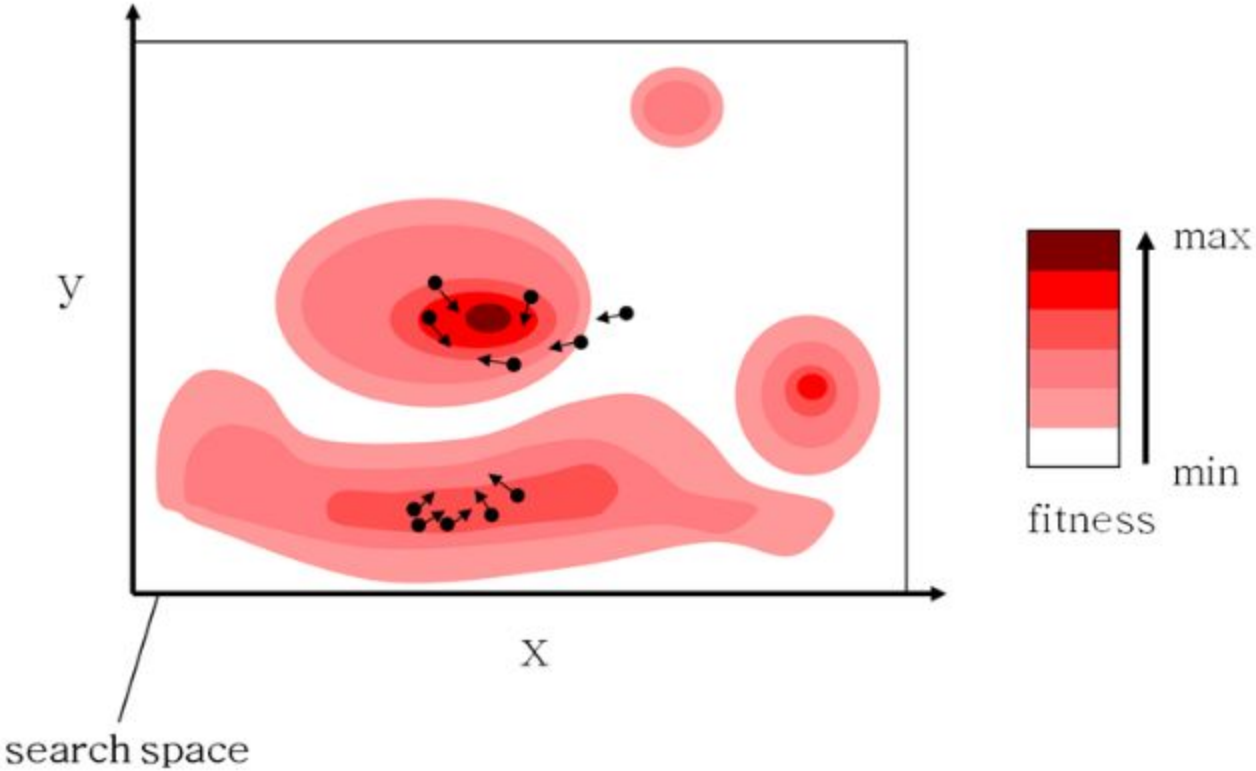
Visualization



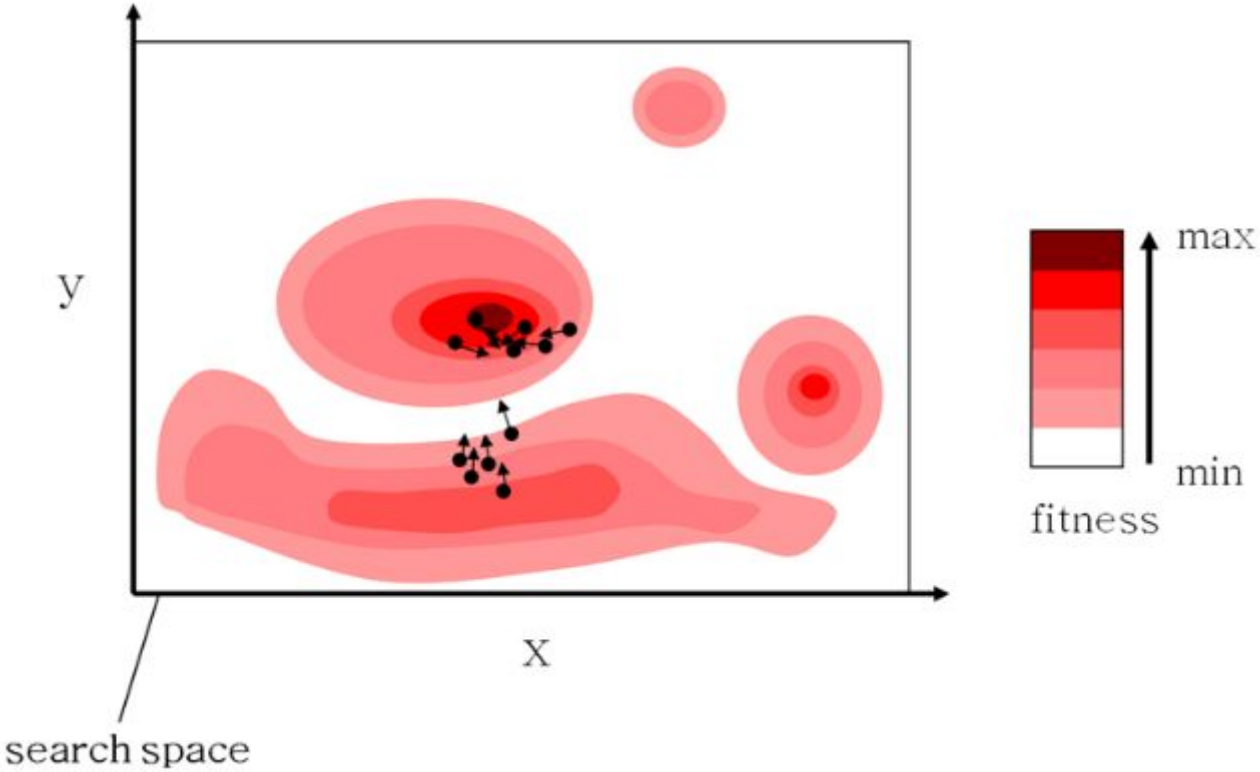
Visualization



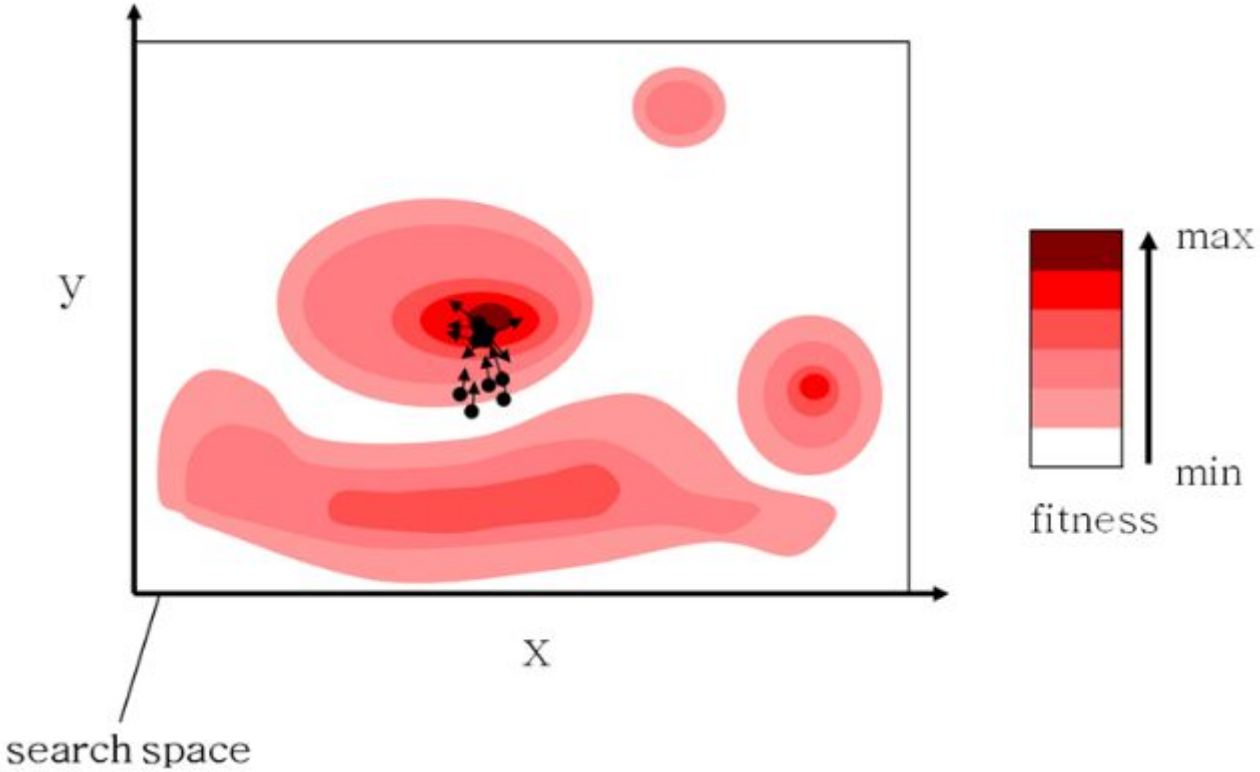
Visualization



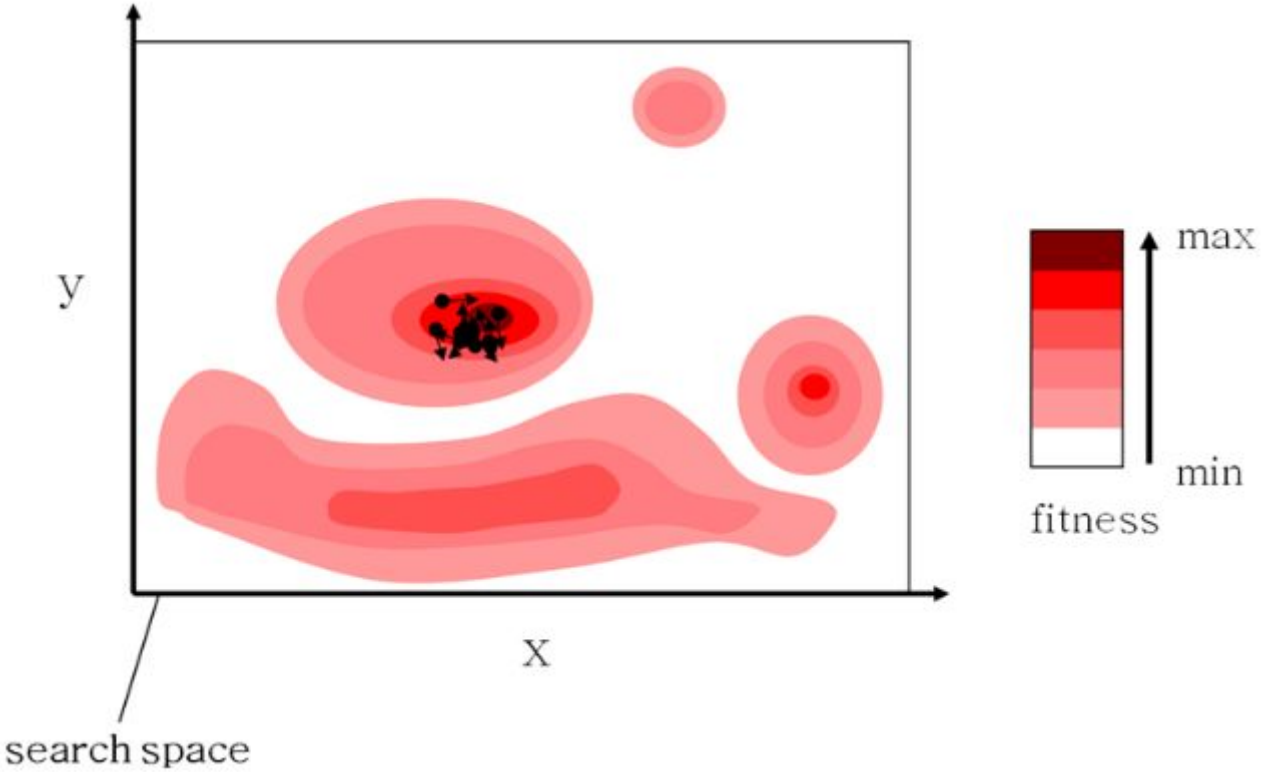
Visualization



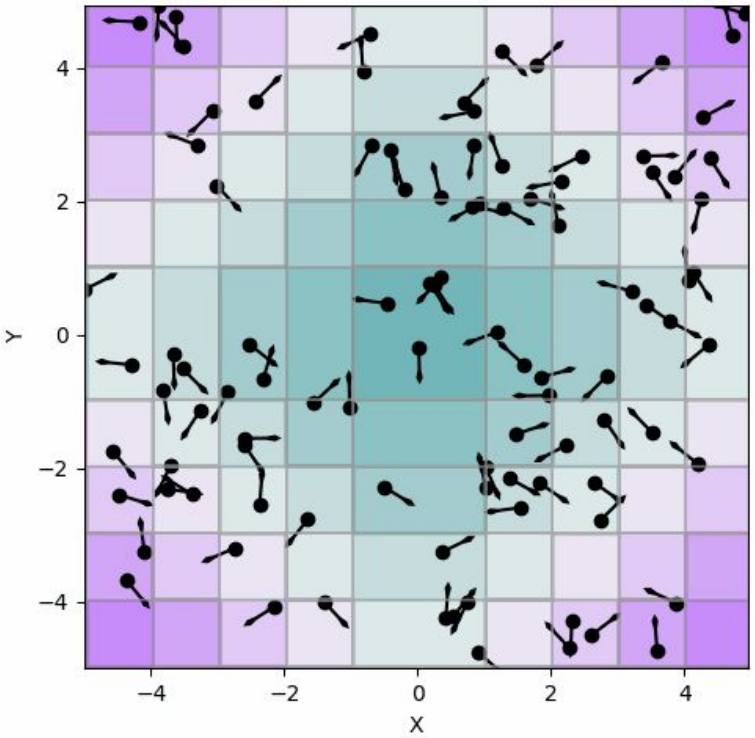
Visualization

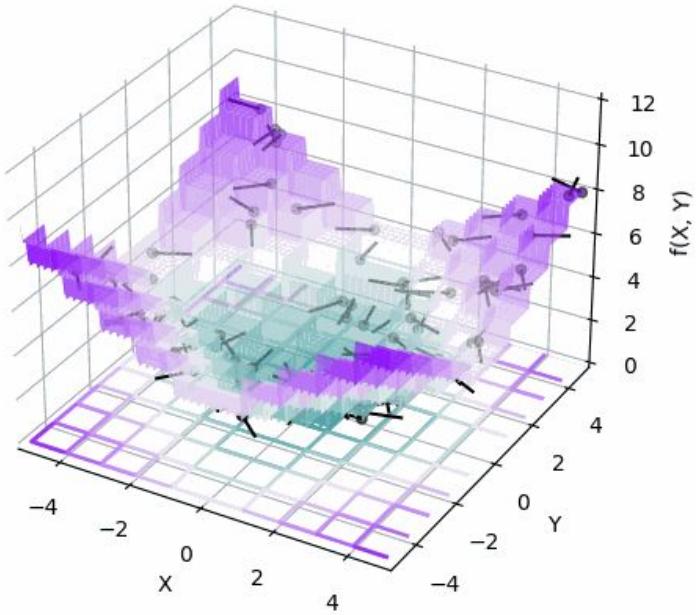


Visualization



Visualization





Applications of PSO

- PSO can basically be applied to any problems where the solutions are **vectors** of **real numbers**
- Some examples include:
  - Ingredients mix optimization
  - Transportation scheduling
  - Bin packing problem
  - etc.

Variants of PSO

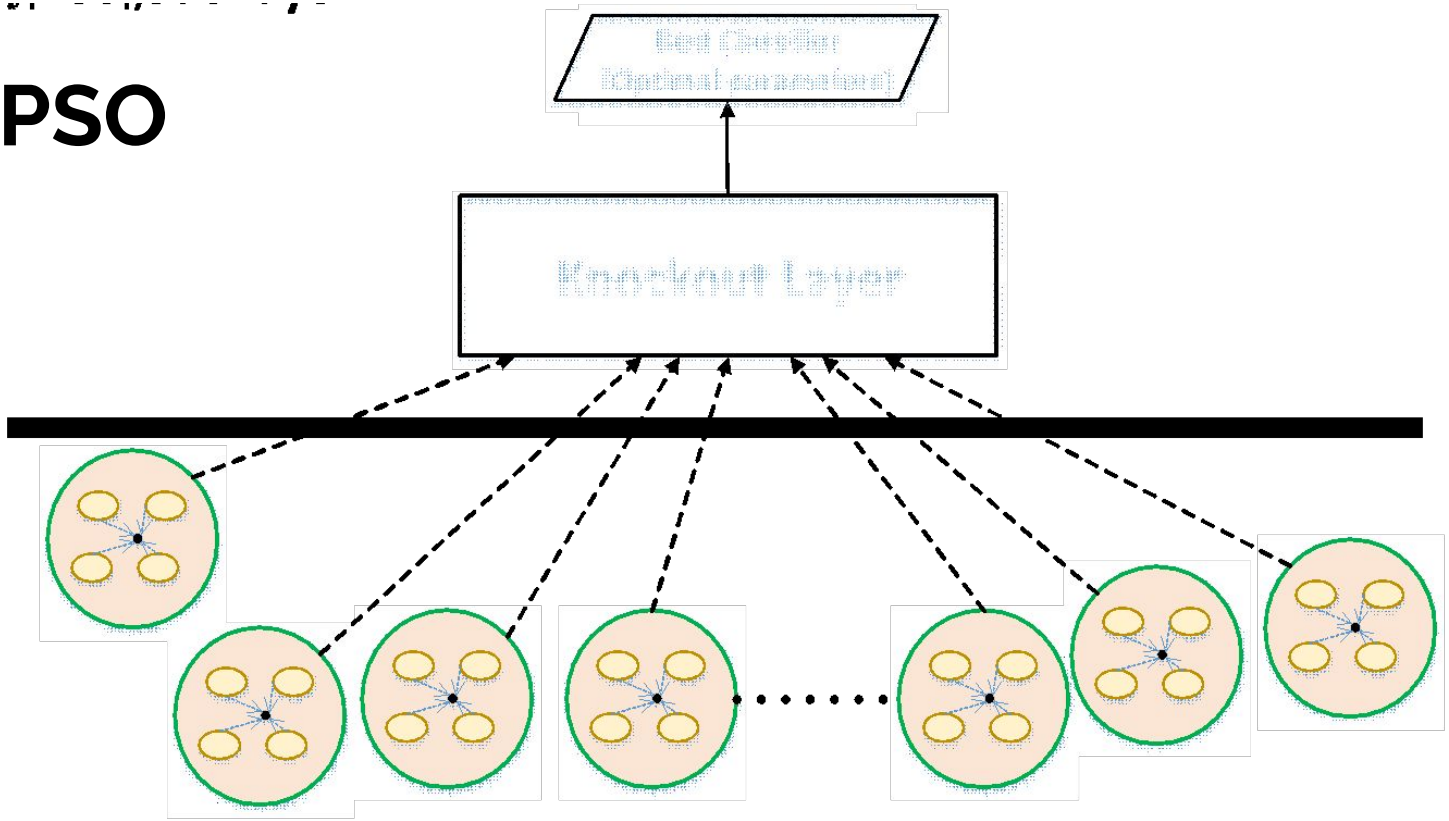
Single-swarm PSO:

- Bare bones PSO (BBPSO) (Kennedy, 2003)
- Accelerated PSO (APSO) (Yang et al. , 2011)
- Rapid-PSO (R-PSO) (Kumar & Kumar, 2017)
- Dll.

Multi-swarm PSO:

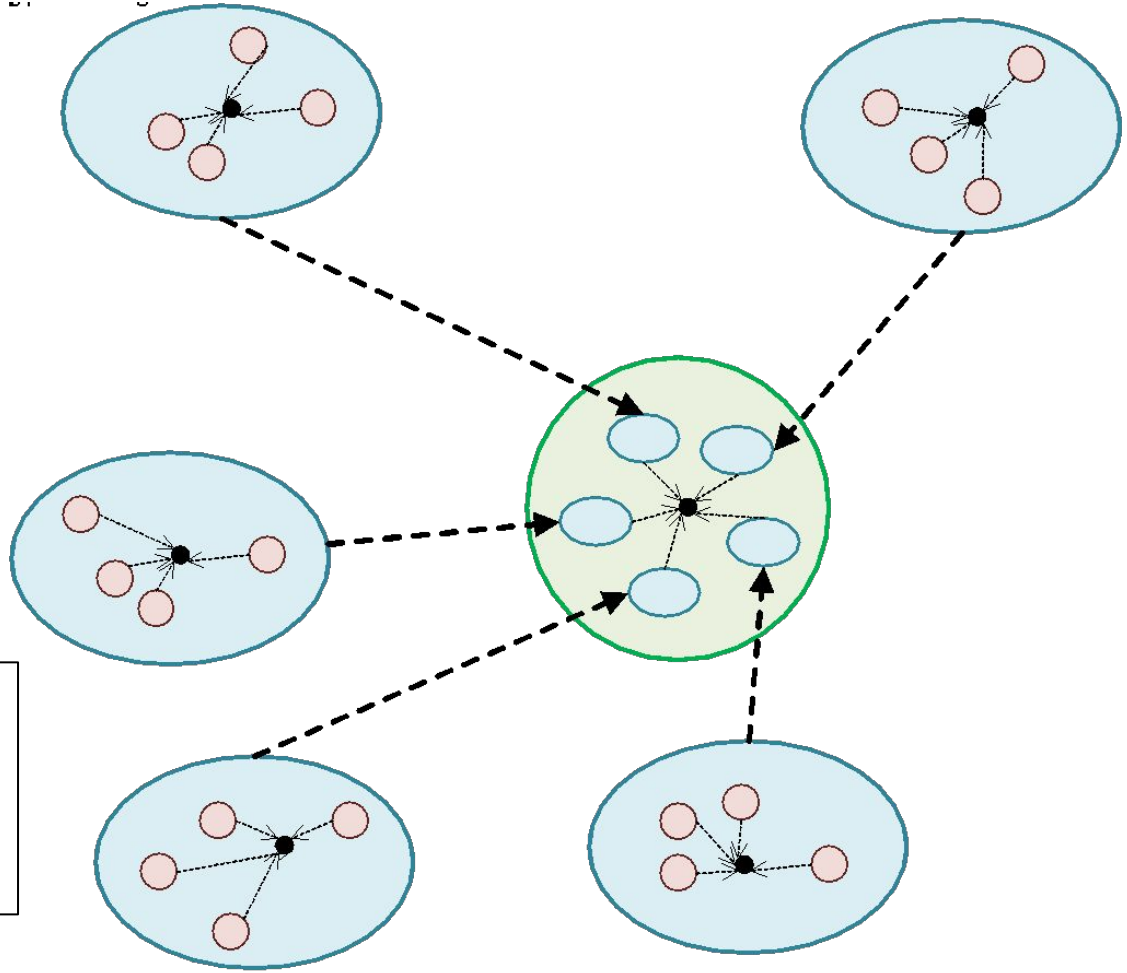
- Wave of Swarm of Particles (WOSP) (Hendtlass, 2005)
- The Dynamic Multi-Swarm-Particle Swarm Optimizer (DMS-PSO) (Zhao et al., 2008)
- *Multi-PSO (Budhi et al., 2018)*
- *Multi-level PSO (ML-PSO) (Budhi, 2020)*
- *Parallel ML-PSO (PML-PSO) (Budhi, 2020)*
- Dll.

Multi-PSO



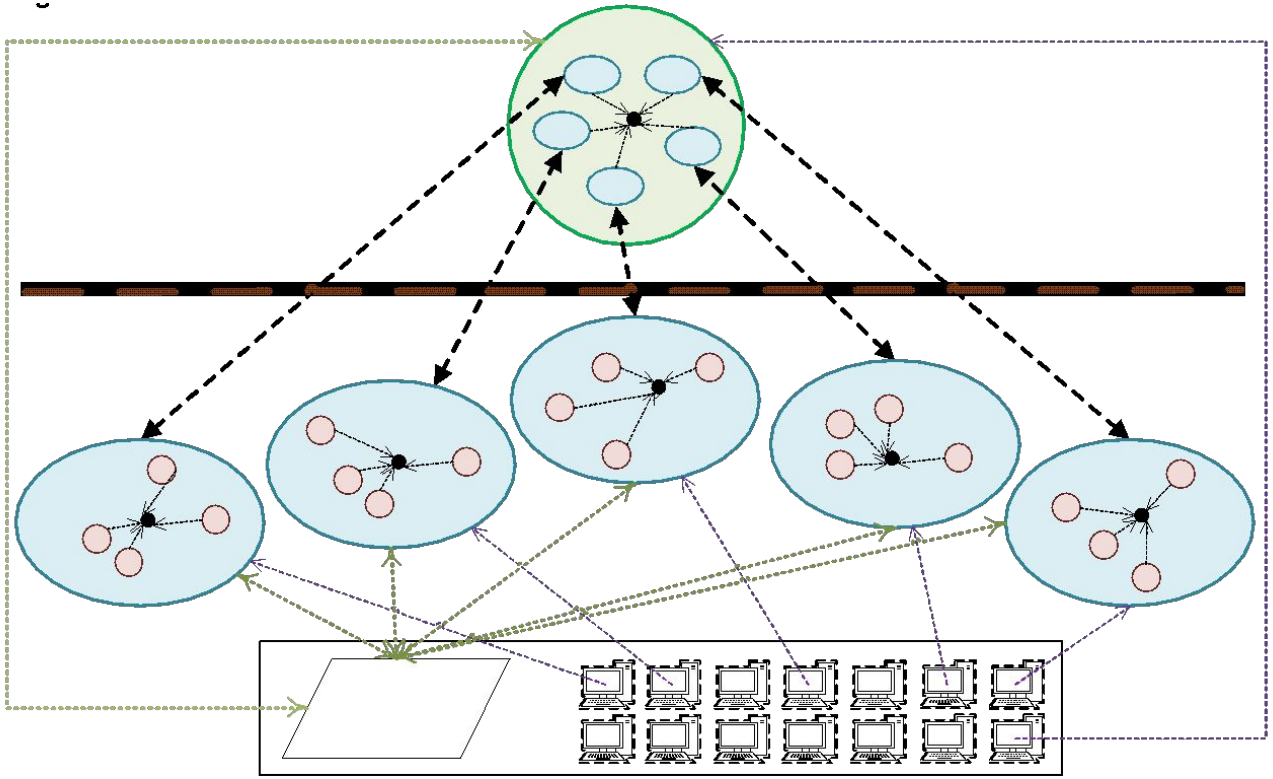
G. S. Budhi, R. Chiong, Z. Hu, I. Pranata, and S. Dhakal, "Multi-PSO based classifier selection and parameter optimisation for sentiment polarity prediction," in *Proceedings of the IEEE Conference on Big Data and Analytics 2018*, Langkawi Island, Malaysia, November 21 – 22, 2018, pp. 68-73, doi: https://doi.org/10.1109/ICBDAA.2018.8629593.

Multi-level PSO



G. S. Budhi, R. Chiong, and S. Dhakal, "Multi-level particle swarm optimisation and its parallel version for parameter optimisation of ensemble models: a case of sentiment polarity prediction," *Cluster Computing,* vol. 23, pp. 3371-3386, 2020, doi: https://doi.org/10.1007/s10586-020-03093-3.

Parallel Multi-level PSO



G. S. Budhi, R. Chiong, and S. Dhakal, "Multi-level particle swarm optimisation and its parallel version for parameter optimisation of ensemble models: a case of sentiment polarity prediction," *Cluster Computing,* vol. 23, pp. 3371-3386, 2020, doi: https://doi.org/10.1007/s10586-020-03093-3.

Swarm Intelligence

Advantages:

- Applicable to wide range of applications
- Easy to understand and implement
  - Each agent is simple, and behaves individually and identically
  - Easy to add or remove individual agents without compromising the swarm
  - Relatively few parameters to adjust
- Relatively efficient computations
  - Scalable to larger applications with more agents

Challenges:

- Convergence to local optima
- Slow convergence

Recent Developments in SI Applications

- U.S. Military is applying SI techniques to control of unmanned vehicles
- NASA is applying SI techniques for planetary mapping
- Medical Research is trying SI based controls for nanobots to fight cancer
- SI techniques are applied to load balancing in telecommunication networks
- Entertainment industry is applying SI techniques for battle and crowd scenes

Thank you for listening